第 4 章練習詳解：執行緒

OS04-E01　共享範圍

1. 在抽象的讀取／加一／寫回模型，兩條執行緒都可先讀到 0，各自加一，再依序寫回 1；因此最後是 1，發生遺失更新（lost update）。但在 C/C++ 中，未同步、非原子的衝突存取構成 data race，行為未定義；上述交錯僅是說明機制的模型，並非對 C/C++ 實際輸出的保證。

   判定 data race 時，要看不同執行緒對同一 memory location 的衝突動作：至少一方修改、至少一項操作非原子，且兩者互不以 happens-before 排序。兩次讀取不互相衝突。

2. Thread-local 儲存讓每條執行緒操作不同的 counter，避免對同一個 counter 的共享寫入競爭；但不再有直接共享的總計值。若需要總和，每個 worker 應在結束前將結果複製到仍然存活、各自獨立的結果槽；主執行緒成功 join 後再讀槽位並加總。

3. 在所有相關存取均遵守同一把 mutex、兩次操作都完成且不溢位的條件下，臨界區的兩次 read-modify-write 互斥執行：0→1→2。鎖外的衝突存取不受這項保證保護。

OS04-E02　並行不保證加速

1. 在題目的固定規模、理想均分及不計額外開銷的模型中，單核心時間比例為循序 0.2、可平行 0.8。8 核執行時間是基準的 `0.2+0.8/8=0.3`，理論加速比上限為 `1/0.3=10/3`，約 3.33 倍。
2. 啟動與同步成本、通訊、負載不均、記憶體頻寬和快取干擾等，可能使相同工作在實際條件下比理想模型慢。模型與基準都必須相同；不同資料配置或快取效應也可能造成模型未涵蓋的結果，不能宣稱任何實測都必定較慢。
3. 循序 5%、可平行 95% 時，理想時間比例為 `0.05+0.95/8=0.16875`，加速比約 `1/0.16875=5.93` 倍。

OS04-E03　取消與清理

1. 非同步取消可能在執行緒持鎖、尚未註冊清理，或共享結構只更新一半時生效，造成鎖無人釋放或不變量被破壞。
2. 延後取消只在規定的 cancellation point 處理請求，讓程式能選擇可清理的邊界；但它不會自動證明資料結構正確。若取消在 `pthread_cond_wait` 生效，該函式先依契約重新取得 mutex，再執行清理處理器。成功送出請求不代表目標已退出。
3. 在註冊處理器前，可暫時停用取消；明定正常、錯誤與取消路徑對 B 和 M 的所有權，確保釋放及解鎖各一次。若共享資料暫時破壞不變量，應避免在此期間到達取消點，或設計 rollback。成功 join 或其他明確的完成協定成立後，才能回收仍可能由 worker 使用的參數。

OS04-E04　M:N 映射與阻塞補償

1. 一個 carrier 阻塞後還有 9 個可執行；固定的 10 個 carrier 全部阻塞後，可執行 carrier 為 0。此時已有 10 條使用者執行緒發出阻塞呼叫，其餘 90 條留在 runtime 的 ready queue，因沒有 carrier 而無法發出各自呼叫。
2. 上述數字依賴「固定 10 個、沒有替代 carrier、每次呼叫真會阻塞 carrier」。若 runtime 能補充第 11 至 20 個 carrier，就可能啟動更多呼叫；非同步 I/O 也可讓許多使用者任務等待完成，而不讓等量的 carrier 在核心內阻塞。使用者層等待和核心 carrier 阻塞是不同狀態。
3. Many-to-one 只有一個 carrier，若它因真正的阻塞呼叫停住，核心無法另行排程該行程的其他使用者執行緒；one-to-one 每條使用者執行緒各有核心實體，一條阻塞不阻止其他 ready 實體被排程。真正平行程度仍受可用核心及 ready 實體數限制。

OS04-E05　建立、join 與 TLS

1. 主執行緒先初始化 input，建立 worker 後維持 Work 存活且避免與 worker 對 output 的寫入衝突。成功 join 後，依使用的執行緒 API 的完成及記憶體同步契約讀取 output＝42。睡 10 ms 不能證明 worker 已經完成；建立成功也不保證 worker 尚未開始。
2. `pthread_join` 成功等待可 join 的目標並取得結果；錯誤碼由函式直接回傳。Windows 的 `WaitForSingleObject` 必須確認 thread handle 已 signaled，timeout 或 `WAIT_FAILED` 不代表完成；完成後仍要 `CloseHandle`，但關閉 handle 不等於終止執行緒。Java 的 `start` 建立新執行流程，直接呼叫 `run` 是目前執行緒上的普通方法呼叫；`join` 若丟出 `InterruptedException`，不能據此認定目標已完成。
3. 返回已結束的 worker 所有之堆疊區域變數指標不安全，因物件生命週期已結束；detach 後立即 free 參數也不安全，worker 仍可能使用它。Detach 需要另一套確保完成與所有權的協定；同步不能讓已失效的物件復活。
4. 兩個 TLS counter 各自 0→1。每個 worker 結束前應將值複製到各自獨立且存活至 join 後的結果槽；主執行緒成功 join 兩者後讀取槽位，計算 1＋1＝2。C++ `thread_local` 物件的儲存只持續到所屬執行緒結束，不能在 join 後直接讀取已結束執行緒的 TLS。執行緒池的 worker 跨工作重用，TLS 也可能跨多個工作存續；每個工作需要的狀態應依協定明確初始化與清理。

OS04-E06　Amdahl、DAG 與 false sharing

1. 理想情況下 `T8=20+80/8=30 ms`，加速比 `100/30=10/3`，效率 `(10/3)/8=5/12`；核心數趨近無限時，加速比上限為 `100/20=5`。若總共再花 10 ms 啟動與同步，則 `T8=40 ms`，加速比為 `100/40=2.5`。
2. 例如 80 個操作各 1 cycle、另 20 個各 10 cycles，基準共 `80+200=280 cycles`。若前 80 個操作理想加速 8 倍而後 20 個不變，改善後為 `10+200=210 cycles`，加速比 `280/210=4/3`。操作數的 80% 並不是時間的 80%。
3. 葉值 1 至 8 相鄰配對得到 `3,7,11,15`，再合併為 `10,26`，最後得到 36。8 個 leaf 加 7 個 merge，共 work＝15；一條最長路徑依序經 leaf、pair、quartet、root，span＝4。任意 P 個處理器的時間滿足 `T_P ≥ max(15/P,4)`；若 15 為同模型的單處理器時間，加速比至多 `15/4`。平均平行度不是「超過 3.75 個處理器必然無收益」的斷點。
4. `a[0]` 與 `a[1]` 是不同的 `int` 物件；在題設沒有其他衝突下，兩個寫入本身不構成同一 memory location 的 data race。但它們位於同一 coherence line，不同核心反覆寫入可能使該 line 在核心間轉移，形成 false sharing。分隔並對齊可降低此干擾，實際改善須量測；padding 也不能修復對同一 counter 的 data race。

OS04-E07　執行緒池與 serial queue 的等待環

1. A 和 B 各占一條 worker 並阻塞等待子工作，A1、B1 雖在佇列中 ready，卻沒有空 worker 可執行。這是資源等待造成的 pool starvation deadlock；資料相依圖為 DAG 不足以排除它。
2. 可使用明確支援 help-join 的 runtime，讓等待者協助執行 ready 子工作；或者避免 pool worker 在同一 pool 內同步等待子工作，改以 continuation 或外部協調。增加 worker 數只能解除這組固定例子，不保證任意深度的巢狀等待安全。
3. Q 是 serial queue，後送入的 block 必須等前一個 block 結束；前一個卻同步等待後者，形成 self-wait。Serial queue 只序列化自身工作，不自動保護不同 queue 的共享物件。
4. 四個區間的 private partial 分別為 `1+2=3`、`3+4=7`、`5+6=11`、`7+8=15`；合併得到 `3+7=10`、`11+15=26`、`10+26=36`。普通共享 `sum+=...` 若無同步不等同 reduction；浮點加法重排還可能改變末位，應依精度要求評估。

OS04-E08　取消、fork 與 signal

1. 不等於。成功送出 cancel 只表示請求已提出；目標可能停用取消或尚未到達延後取消點。以成功 join 或其他明確的完成協定確認結束後，才能回收 worker 仍可能使用的參數。
2. `pthread_cond_wait` 原子地釋放 M 並等待；取消在此等待中生效時，依契約先重新取得 M，接著按 LIFO 執行處理器：先 free B 一次，再 unlock M 一次。正常路徑也要正確配對 pop／清理，避免重複釋放。
3. 不一定。只解鎖可能把半更新的錯誤狀態暴露給下一個持鎖者。必須在不變量穩定時允許取消，或提供真正的 rollback；任意點生效的非同步取消特別難保證此性質。
4. Fork 子行程只保留呼叫 fork 的 T1；T2 不存在，但其持有的鎖狀態可能仍留在複製的記憶體中，子行程若嘗試取得該鎖可能無法前進。子行程在 exec 前須遵守相關 async-signal-safe 限制。Fork 後繼承父行程的 signal disposition 與 T1 的 signal mask，但子行程的 pending signals 初始為空。Disposition 通常由行程內執行緒共享，mask 則各執行緒獨立；process-directed signal 不等於每條執行緒都執行一次 handler。

OS04-E09　Windows 與 Linux 固定版本案例

1. `CreateProcessW` 已建立一條初始執行緒，再呼叫一次 `CreateThread` 後共有兩條執行緒與兩個 TEB；各自有執行緒執行狀態及堆疊，共享同一行程的位址空間及行程層級資源。兩者各有對應的 ETHREAD、KTHREAD 與 TEB。
2. 在題設 Linux 3.2 一個原 task、呼叫成功且無額外 task 的模型中，一般 fork 後共有 2 個 task、2 個 mm 與各自的頁表；檔案描述符表的 files 結構複製成兩份，但其中對同一已開啟檔案的項目仍可能引用同一 open file description。指定旗標的同群組 clone 則有 2 個 task 和各自的 kernel stack，但共用 1 個 mm／頁表、1 個 files 結構與 1 個 sighand 結構；同群組共享 TGID，task 身分仍不同。
3. 不必然。缺少 `CLONE_THREAD` 即使共享 VM／FS／FILES／SIGHAND，也不具有同一 thread group 的語意。Linux 3.2 中 `CLONE_THREAD` 要求 `CLONE_SIGHAND`，而後者要求 `CLONE_VM`；建立完整 pthread 還涉及 TLS 和終止等其他機制，不能只憑列出的旗標等同。
4. 不會。共用 mm 只是共用位址空間；對同一物件的衝突存取仍要遵守相應同步、記憶體順序與生命週期規則。

OS04-E10　排程實體與 CPU 份額

1. Many-to-one 有兩個可執行 carrier，各得 1/2；因此 P＝1/2、Q＝1/2。One-to-one 有五個核心實體，各得 1/5；P 的三條合計 3/5＝60%，Q 的兩條合計 2/5＝40%。
2. 若排程器先對行程群組平均分配 CPU，再在群組內分配，兩個群組可能仍各得 1/2；權重不同或部分執行緒阻塞也會改變結果。此計算只適用於題目給定的單 CPU、持續可執行與等份額假設。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 4. Linux 3.2 fork.c; pthread_join(3); pthread_cond_wait(3p); C++ thread storage; OpenMP reduction.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study